# Section A – Debugging Questions

## 1. Non-Virtual Destructor Memory Leak in Polymorphic Deletion

1. Defect and reason:

The defect is in Line 7, where the Shape destructor is not declared virtual.

When delete shapePtr; is executed, shapePtr is a Shape*. Without a virtual destructor, deleting the object through this base pointer does not correctly invoke the derived destructor.

As a result, ComplexPolygon's destructor does not release the dynamically allocated vertexArray.

2. C++ standard rule:

Deleting a derived object through a base class pointer when the base class destructor is non-virtual results in undefined behavior.

3. Correction:

Declare the base class destructor as virtual.

The virtual destructor allows the program to invoke the derived destructor first and then the base destructor.

4. Corrected C++ code:

```
#include <iostream>
using namespace std;

class Shape {
public:
   Shape() {
      cout << "Shape created." << endl;
   }

   virtual ~Shape() {
      cout << "Shape destroyed." << endl;
   }

   virtual void render() const {
      cout << "Rendering generic shape." << endl;
```

```
    }
};

class ComplexPolygon : public Shape {
private:
    double *vertexArray;
    int vertexCount;

public:
    ComplexPolygon(int count) : vertexCount(count) {
        vertexArray = new double[vertexCount * 2];
        cout << "Allocated " << vertexCount
             << " vertices on heap." << endl;
    }

    ~ComplexPolygon() {
        cout << "Deallocating polygon vertex heap memory."
             << endl;
        delete[] vertexArray;
    }

    void render() const override {
        cout << "Rendering polygon with "
             << vertexCount << " vertices." << endl;
    }
};

int main() {
    Shape *shapePtr = new ComplexPolygon(1000);

    shapePtr->render();

    delete shapePtr;

    return 0;
}
```

Output:

```
Shape created.
Allocated 1000 vertices on heap.
Rendering polygon with 1000 vertices.
```

Deallocating polygon vertex heap memory.
Shape destroyed.

Explanation: The vtable supports virtual function dispatch, including the correct destructor call. The derived destructor executes before the base destructor, ensuring allocated memory is released.

## 2. Inverted Exception Catch Handlers & Unreachable Code

1. Defect:

The base class handler catch (const BankingException &be) is placed before the derived class handler catch (const InsufficientFundsException &ife).

The compiler may generate a warning that the derived exception will be caught by the earlier handler.

2. Explanation:

C++ checks catch handlers from top to bottom.

Since InsufficientFundsException inherits from BankingException, the base class handler can also catch the derived exception. Therefore, the specific handler is never reached.

3. Correct rule:

Always place derived class exception handlers before base class exception handlers.

4. Corrected code:

```
#include <iostream>
#include <string>
using namespace std;

class BankingException {
protected:
   string msg;

public:
   BankingException(string m) : msg(m) {}

   virtual string getMessage() const {
      return msg;
   }
};
```

```
class InsufficientFundsException : public BankingException {
private:
   double shortfall;

public:
   InsufficientFundsException(string m, double sf)
      : BankingException(m), shortfall(sf) {}

   double getShortfall() const {
      return shortfall;
   }
};

void processWithdrawal(double balance, double amount) {
   if (amount > balance) {
      throw InsufficientFundsException(
         "Account overdrawn!", amount - balance
      );
   }
}

int main() {
   try {
      processWithdrawal(500.0, 750.0);
   }
   catch (const InsufficientFundsException &ife) {
      cout << "Shortfall of: $"
           << ife.getShortfall() << endl;
   }
   catch (const BankingException &be) {
      cout << "General Banking Error: "
           << be.getMessage() << endl;
   }
   catch (...) {
      cout << "Unknown error occurred." << endl;
   }

   return 0;
}
```

Output:

Shortfall of: $250

## 3. Exception Rethrowing Slicing Bug (throw e; vs. throw;)

1. Defect and current output:

The error occurs at Line 32:

throw e;

This creates and throws a copy using the static type DatabaseError. The derived type TimeoutError is sliced.

Therefore, the TimeoutError handler in main() is bypassed.

Current output:

[AUDIT LOG]: Intercepted DB error.
Client caught generic DatabaseError: Query timed out connecting to shard 3

2. Difference between throw e; and throw;:

| **throw e;** | **throw;** |
|---|---|
| Throws a copy of the caught expression | Rethrows the original exception |
| Can cause object slicing | Preserves the original derived type |
| May lose derived attributes | Retains derived attributes |

3. Correction:

Replace throw e; with throw;.

4. Corrected C++ code:

```
#include <iostream>
#include <exception>
#include <string>
using namespace std;

class DatabaseError : public exception {
```

```
protected:
    string errorDetails;

public:
    DatabaseError(string msg) : errorDetails(msg) {}

    const char* what() const noexcept override {
        return errorDetails.c_str();
    }
};

class TimeoutError : public DatabaseError {
public:
    int timeoutMs;

    TimeoutError(string msg, int ms)
        : DatabaseError(msg), timeoutMs(ms) {}
};

void executeQuery() {
    throw TimeoutError(
        "Query timed out connecting to shard 3", 5000
    );
}

void intermediateAuditor() {
    try {
        executeQuery();
    }
    catch (const DatabaseError &e) {
        cout << "[AUDIT LOG]: Intercepted DB error."
             << endl;

        throw;
    }
}

int main() {
    try {
        intermediateAuditor();
    }
    catch (const TimeoutError &te) {
        cout << "Client caught TimeoutError! Duration: "
```

```
            << te.timeoutMs << " ms" << endl;
    }
    catch (const DatabaseError &de) {
        cout << "Client caught generic DatabaseError: "
            << de.what() << endl;
    }

    return 0;
}
```

Output:

```
[AUDIT LOG]: Intercepted DB error.
Client caught TimeoutError! Duration: 5000 ms
```

## 4. Attempting to Instantiate an Incomplete Abstract Class

1. Compilation error:

The error occurs at Line 25, when the program tries to create a VideoPacket object.

The class does not implement the pure virtual function getPacketSize().

Typical compiler error:

error: cannot declare variable 'packet' to be of abstract type 'VideoPacket'

2. Abstract class:

A class containing at least one pure virtual function is called an abstract class.

An abstract class cannot be directly instantiated.

3. When does a derived class become concrete?

A derived class becomes concrete when it implements all inherited pure virtual functions.

4. Corrected C++ code:

```
#include <iostream>
using namespace std;

class NetworkPacket {
public:
    virtual ~NetworkPacket() {}
```

```
    virtual void serialize() = 0;
    virtual int getPacketSize() = 0;
};

class VideoPacket : public NetworkPacket {
private:
    int frameID;

public:
    VideoPacket(int id) : frameID(id) {}

    void serialize() override {
        cout << "Serializing H.264 Video Frame: "
             << frameID << endl;
    }

    int getPacketSize() override {
        return 1024;
    }
};

int main() {
    VideoPacket packet(1042);

    packet.serialize();

    cout << "Packet Size: "
         << packet.getPacketSize() << endl;

    return 0;
}
```

Output:

```
Serializing H.264 Video Frame: 1042
Packet Size: 1024
```

## 5. Pass-by-Value Object Slicing Suppressing Dynamic Binding

1. Reason for incorrect output:

The function triggerEncounter(Entity e) accepts an object by value.

When a Dragon object is passed, only its Entity portion is copied. This is called object slicing.

Therefore, Entity::attack() is executed instead of Dragon::attack().

2. Role of vptr:

- The vptr points to the virtual table containing virtual function information.
- Passing by value creates a separate base class object.
- The copied object has the base class type, so dynamic dispatch to the original Dragon implementation is lost.
- Passing by reference preserves the original derived object and its dynamic type.

3. Correction:

Change the function parameter from Entity e to const Entity &e.

4. Corrected C++ code:

```
#include <iostream>
using namespace std;

class Entity {
public:
   virtual ~Entity() {}

   virtual void attack() const {
      cout << "Entity performs generic scratch."
           << endl;
   }
};

class Dragon : public Entity {
public:
   void attack() const override {
      cout << "Dragon breathes scorching fire!"
           << endl;
   }
};

void triggerEncounter(const Entity &e) {
   e.attack();
}
```

```
int main() {
   Dragon boss;

   triggerEncounter(boss);

   return 0;
}
```

Output:

Dragon breathes scorching fire!

# Section B – Pseudocode Writing Questions

## 1. Universal Payment Gateway with Runtime Polymorphism

1. Structured pseudocode:

```
ABSTRACT CLASS PaymentGateway

   ABSTRACT METHOD Authorize(amount AS REAL)
      RETURNS BOOLEAN

   ABSTRACT METHOD Capture()
      RETURNS BOOLEAN

   VIRTUAL DESTRUCTOR()
END CLASS


CLASS CreditCardGateway EXTENDS PaymentGateway

   METHOD Authorize(amount AS REAL) RETURNS BOOLEAN
      VALIDATE card length = 16 digits
      VALIDATE CVV = 3 digits

      IF validation successful THEN
         PRINT "Credit Card Authorized"
         RETURN TRUE
      ELSE
         RETURN FALSE
      END IF
```

```
    END METHOD

    METHOD Capture() RETURNS BOOLEAN
        PRINT "Credit Card Payment Captured"
        RETURN TRUE
    END METHOD
END CLASS


CLASS CryptoGateway EXTENDS PaymentGateway

    METHOD Authorize(amount AS REAL) RETURNS BOOLEAN
        VERIFY wallet hash starts with "0x"
        VALIDATE gas fee

        IF validation successful THEN
            PRINT "Crypto Payment Authorized"
            RETURN TRUE
        ELSE
            RETURN FALSE
        END IF
    END METHOD

    METHOD Capture() RETURNS BOOLEAN
        PRINT "Crypto Payment Captured"
        RETURN TRUE
    END METHOD
END CLASS


BEGIN MAIN

    DECLARE gatewayPtr AS POINTER TO PaymentGateway

    gatewayPtr <- NEW CryptoGateway()

    IF gatewayPtr->Authorize(1500.00) = TRUE THEN
        CALL gatewayPtr->Capture()
    ELSE
        PRINT "Payment Failed"
    END IF

    DELETE gatewayPtr
```

END MAIN

2. Dry run – Cryptocurrency payment of $1,500:

| Step | Operation | Result |
|---|---|---|
| 1 | Create CryptoGateway object | Object created |
| 2 | Assign to PaymentGateway pointer | Upcasting |
| 3 | Authorize(1500.00) | Wallet and gas fee validated |
| 4 | Capture() | Payment captured |
| 5 | Delete gatewayPtr | Object deallocated |

Output:

Crypto Payment Authorized
Crypto Payment Captured

Explanation: The base class pointer calls the derived class methods through runtime polymorphism. The virtual destructor ensures safe deletion.

## 2. Resilient Banking Engine with Nested Try-Catch & Rethrowing

1. Structured pseudocode:

```
CLASS TransactionException
   DECLARE reason AS STRING

   CONSTRUCTOR(message)
      reason <- message
   END CONSTRUCTOR
END CLASS


CLASS AccountFrozenException EXTENDS TransactionException
   CONSTRUCTOR(message)
      CONSTRUCTOR TransactionException(message)
```

```
    END CONSTRUCTOR
END CLASS


CLASS DailyLimitExceededException EXTENDS TransactionException
    DECLARE excessAmount AS REAL

    CONSTRUCTOR(message, excess)
        CONSTRUCTOR TransactionException(message)
        excessAmount <- excess
    END CONSTRUCTOR
END CLASS


FUNCTION DebitLedger(amount, isFrozen)

    IF isFrozen = TRUE THEN
        THROW NEW AccountFrozenException(
            "Account is frozen"
        )
    END IF

    IF amount > 50000 THEN
        THROW NEW DailyLimitExceededException(
            "Daily limit exceeded",
            amount - 50000
        )
    END IF

    PRINT "Debit Successful"

END FUNCTION


FUNCTION AuditTransactionWrapper(amount, isFrozen)

    TRY
        CALL DebitLedger(amount, isFrozen)

    CATCH (e AS TransactionException)
        PRINT "Security Audit Log:", e.reason
        RETHROW
    END TRY
```

```
END FUNCTION


BEGIN MAIN

   TRY
      CALL AuditTransactionWrapper(1000, TRUE)

   CATCH (e AS AccountFrozenException)
      PRINT "Transaction Failed: Account Frozen"

   CATCH (e AS DailyLimitExceededException)
      PRINT "Daily Limit Exceeded"
      PRINT "Excess Amount:", e.excessAmount

   CATCH (e AS TransactionException)
      PRINT "General Transaction Error"

   END TRY

END MAIN
```

2. Dry-run trace table:

A withdrawal of $1,000 is attempted on a frozen account.

| Step | Execution | Result |
|---|---|---|
| 1 | Main calls AuditTransactionWrapper() | Enters function |
| 2 | Wrapper calls DebitLedger() | Enters function |
| 3 | Checks account status | Account is frozen |
| 4 | Throws AccountFrozenException | Control transfers to catch |
| 5 | Wrapper catches TransactionException | Security log printed |

| | | |
|---|---|---|
| 6 | RETHROW executes | Original exception is passed upward |
| 7 | Main catches AccountFrozenException | Displays failure message |

Output:

Security Audit Log: Account is frozen
Transaction Failed: Account Frozen

Explanation: The intermediate function records the error and rethrows it. The main function handles the specific exception.

## 3. Polymorphic Game Engine with Virtual Destructors

1. Structured pseudocode:

```
ABSTRACT CLASS GameObject

   ABSTRACT METHOD Update(deltaTime AS REAL)

   ABSTRACT METHOD Render()

   VIRTUAL DESTRUCTOR()
      PRINT "GameObject destroyed"
   END DESTRUCTOR

END CLASS


CLASS ParticleSystem EXTENDS GameObject

   DECLARE particleCoordinates AS POINTER TO REAL
   DECLARE particleCount AS INTEGER

   CONSTRUCTOR(count)
      particleCount <- count
      particleCoordinates <- NEW REAL[count]
      PRINT "Particle memory allocated"
   END CONSTRUCTOR

   OVERRIDE METHOD Update(deltaTime AS REAL)
      PRINT "Updating particles"
```

```
    END METHOD

    OVERRIDE METHOD Render()
        PRINT "Rendering particles"
    END METHOD

    DESTRUCTOR()
        DELETE[] particleCoordinates
        PRINT "Particle memory released"
        PRINT "ParticleSystem destroyed"
    END DESTRUCTOR

END CLASS


BEGIN MAIN

    DECLARE sceneEntities AS ARRAY OF POINTER TO GameObject

    sceneEntities[0] <- NEW ParticleSystem(100)

    FOR EACH entity IN sceneEntities
        CALL entity->Update(0.016)
        CALL entity->Render()
    END FOR

    FOR EACH entity IN sceneEntities
        DEALLOCATE entity
    END FOR

END MAIN
```

2. Cleanup phase and destructor order:

When DEALLOCATE is called through a GameObject* pointer:

1. ParticleSystem destructor executes first.
2. Dynamically allocated particle memory is released.
3. GameObject destructor executes next.
4. The object is completely destroyed.

Output:

Particle memory allocated
Updating particles
Rendering particles
Particle memory released
ParticleSystem destroyed
GameObject destroyed

Explanation: A virtual destructor ensures that both derived and base destructors execute correctly when deleting an object through a base class pointer.

## 4. File Stream Pipeline with Stack Unwinding & Catch-All

1. Structured pseudocode:

```
CLASS ResourceGuard

    DECLARE fileHandle AS INTEGER

    CONSTRUCTOR(fileName)
        fileHandle <- OPEN fileName
        PRINT "File resource acquired"
    END CONSTRUCTOR

    DESTRUCTOR()
        CLOSE fileHandle
        PRINT "File resource released"
    END DESTRUCTOR

END CLASS


CLASS CRCValidationException

    DECLARE expectedChecksum AS INTEGER
    DECLARE calculatedChecksum AS INTEGER

    CONSTRUCTOR(expected, calculated)
        expectedChecksum <- expected
        calculatedChecksum <- calculated
    END CONSTRUCTOR

END CLASS
```

```
FUNCTION IngestDataStream(fileName)

    DECLARE resource AS ResourceGuard(fileName)

    READ data blocks
    CALCULATE checksum

    IF expectedChecksum != calculatedChecksum THEN
        THROW NEW CRCValidationException(
            expectedChecksum, calculatedChecksum
        )
    END IF

    PRINT "File validation successful"

END FUNCTION


BEGIN MAIN

    TRY
        CALL IngestDataStream("data.txt")

    CATCH (e AS CRCValidationException)
        PRINT "CRC Validation Failed"
        PRINT "Expected:", e.expectedChecksum
        PRINT "Calculated:", e.calculatedChecksum

    CATCH (...)
        PRINT "Unknown error occurred"

    END TRY

END MAIN
```

2. Dry-run trace – CRC mismatch:

| Step | Execution | Result |
|---|---|---|
| 1 | Main calls IngestDataStream() | Function starts |

| | | |
|---|---|---|
| 2 | ResourceGuard constructor executes | File opened |
| 3 | Data blocks are read | Checksum calculated |
| 4 | CRC mismatch occurs | Exception thrown |
| 5 | Stack unwinding begins | Local objects destroyed |
| 6 | ResourceGuard destructor executes | File closed |
| 7 | Main catches CRC exception | Error displayed |

Output:

File resource acquired
File resource released
CRC Validation Failed
Expected: 1234
Calculated: 5678

Explanation: Stack unwinding automatically destroys local stack objects when an exception is thrown. The ResourceGuard destructor closes the file before the catch block executes. The catch-all handler handles any other unexpected exception.

## 5. Scientific Telemetry Calibration with noexcept Contracts

1. Structured pseudocode:

```
CLASS OutOfBoundsException

   DECLARE temperature AS REAL

   CONSTRUCTOR(temp)
      temperature <- temp
   END CONSTRUCTOR

END CLASS


FUNCTION CalibrateThermalSensors(rawVoltage)
```

```
    NOEXCEPT(FALSE)

    CALCULATE temperature FROM rawVoltage

    IF temperature < -273.15 OR temperature > 1500.0 THEN
        THROW NEW OutOfBoundsException(temperature)
    END IF

    RETURN temperature

END FUNCTION


FUNCTION CommitTelemetry(finalTemp) NOEXCEPT

    TRY
        COMMIT finalTemp TO avionics hardware bus
        PRINT "Telemetry committed successfully"

    CATCH (...)
        PRINT "Hardware error suppressed"

    END TRY

END FUNCTION


BEGIN MAIN

    TRY
        DECLARE rawVoltage AS REAL
        DECLARE finalTemp AS REAL

        rawVoltage <- READ INPUT

        finalTemp <- CalibrateThermalSensors(rawVoltage)

        CALL CommitTelemetry(finalTemp)

    CATCH (e AS OutOfBoundsException)
        PRINT "Temperature out of bounds:"
        PRINT e.temperature
        PRINT "Calibration failed"
```

```
    CATCH (...)
        PRINT "Unknown telemetry error"

    END TRY

END MAIN
```

2. Dry-run trace – Faulty temperature reading of −300.0°C:

| Step | Execution | Result |
|---|---|---|
| 1 | Main reads raw voltage | Input received |
| 2 | Calls CalibrateThermalSensors() | Temperature calculated |
| 3 | Temperature = −300.0°C | Below −273.15°C |
| 4 | OutOfBoundsException thrown | Control transfers to catch |
| 5 | Main catches exception | Displays error |
| 6 | CommitTelemetry() | Not executed |

Output:

```
Temperature out of bounds:
-300
Calibration failed
```

Explanation:

- noexcept(false) means the calibration function is allowed to throw exceptions.
- noexcept means the commitment function must not allow exceptions to escape.
- The catch-all block inside CommitTelemetry() handles and suppresses internal exceptions.
- Since calibration fails, the commitment function is not called.